qual_summary.py 사용법 설명서

> lab11 · 생물정보학실험 대상 파일: `src/qual_summary.py`

---

1. 프로그램의 목적과 기능

`qual_summary.py`는 **FASTQ 파일에 들어 있는 모든 read의 평균 품질 점수를 계산하는 프로그램**입니다.

시퀀싱 기계는 DNA 염기(A, C, G, T)를 읽을 때마다 "이 염기를 얼마나 확실하게 읽었는지"를 **품질 점수(quality score)**로 함께 기록합니다. 이 프로그램은 그 점수들을 모아서 "이 시퀀싱 데이터는 전체적으로 얼마나 믿을 만한가?"를 숫자 하나로 보여 줍니다.

프로그램이 하는 일은 다음 세 단계입니다.

1. FASTQ 파일을 읽어서 read를 하나씩 꺼낸다. → `read_fastq()`
2. read마다 평균 품질 점수를 계산한다. → `mean_quality()`
3. 모든 read의 평균 품질을 다시 평균 내서 화면에 출력한다.

---

2. 실행 명령어와 사용 방법

`lab11` 폴더에서 아래처럼 실행합니다.

```
cd lab11
python3 src/qual_summary.py 〈FASTQ 파일 경로〉
```

예시:

```
python3 src/qual_summary.py data/SRR1553607_1.fastq
python3 src/qual_summary.py data/SRR1553607_2.fastq
```

- 입력 파일은 **하나만** 넣습니다. (`sys.argv[1]`로 받습니다.)
- 출력은 파일로 저장되지 않고 **화면에 한 줄**로 나옵니다.
- 결과를 파일로 남기고 싶다면 리다이렉션(`>`)을 쓰면 됩니다.

```
python3 src/qual_summary.py data/SRR1553607_1.fastq > doc/qual_1.txt
```

> **주의:** 파일 이름을 빼먹고 실행하면 `IndexError`가 납니다. 반드시 FASTQ 파일 경로를 함께 적어 주세요.

---

3. `read_fastq()` 함수의 역할

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))
    return records
```

FASTQ 파일은 **read 하나가 항상 4줄**로 이루어져 있습니다.

줄	내용	예시
1번째 줄	`@`로 시작하는 read 이름	`@SRR1553607.1 1 length=101`
2번째 줄	염기 서열	`GTTAGCGTTGTTGATCGC...`
3번째 줄	`+` (구분선)	`+SRR1553607.1 1 length=101`
4번째 줄	품질 문자열 (염기마다 문자 1개)	`BB@FFFFFHHHHHJJJJ...`

`read_fastq()`는 이 규칙을 이용합니다.

- `f.readlines()`로 파일의 모든 줄을 리스트로 읽습니다.
- `range(0, len(lines), 4)`로 **4줄씩 건너뛰며** read의 시작 줄을 찾습니다.
- `lines[i]` → 이름 줄. `.rstrip()`으로 줄바꿈을 지우고 `[1:]`로 맨 앞의 `@`를 뗍니다.
- `lines[i + 1]` → 서열
- `lines[i + 3]` → 품질 문자열 (3번째 줄 `+`는 쓰지 않으므로 건너뜁니다)
- `(이름, 서열, 품질)`을 튜플로 묶어 `records` 리스트에 넣고, 마지막에 돌려줍니다.

> **참고:** 이 함수는 파일 전체를 한 번에 메모리에 올립니다. 실습용 파일(약 20만 read)에는 문제없지만, 아주 큰 파일에서는 메모리를 많이 쓸 수 있습니다.

4. `mean_quality()` 함수의 역할

```
def mean_quality(qual):
    scores = []
    for ch in qual:
        scores.append(ord(ch) - 33)
    return sum(scores) / len(scores)
```

품질 문자열 하나를 받아서 **그 read의 평균 품질 점수**를 돌려줍니다.

- 품질 문자열의 문자를 하나씩(`for ch in qual`) 꺼냅니다.
- `ord(ch) - 33`으로 문자를 숫자 점수로 바꿉니다. (이유는 5장에서 설명)
- 바꾼 점수를 `scores` 리스트에 모읍니다.
- `sum(scores) / len(scores)` = 점수의 합 ÷ 염기 개수 = **평균**

---

5. Phred+33 인코딩과 평균 품질 계산 과정

5-1. Phred 품질 점수란?

Phred 점수(Q)는 **염기를 잘못 읽었을 확률**을 나타내는 점수입니다. 점수가 높을수록 정확합니다.

Phred 점수 (Q)	틀릴 확률	정확도
10	1/10	90%
20	1/100	99%
30	1/1,000	99.9%
40	1/10,000	99.99%

보통 **Q30 이상이면 품질이 좋다**고 봅니다.

5-2. 왜 숫자가 아니라 문자로 저장할까?

점수를 `37 39 41 ...`처럼 숫자로 쓰면 염기 하나에 글자가 2개 이상 필요하고, 서열과 길이도 맞지 않습니다. 그래서 **점수 하나를 문자 하나로 바꿔서** 저장합니다. 이때 쓰는 규칙이 **Phred+33**입니다.

> **Phred+33 규칙:** 품질 문자 = (점수 + 33)에 해당하는 ASCII 문자 거꾸로, **점수 = 문자의 ASCII 번호 − 33**

33을 더하는 이유는 ASCII 0~32번이 공백·줄바꿈 같은 "눈에 안 보이는 문자"이기 때문입니다. 33번(`!`)부터는 눈에 보이는 문자라서 파일에 안전하게 쓸 수 있습니다.

파이썬의 `ord()` 함수는 문자의 ASCII 번호를 알려 줍니다.

품질 문자	`ord(문자)`	− 33	Phred 점수
`!`	33	− 33	**0**
`#`	35	− 33	**2**
`@`	64	− 33	**31**
`B`	66	− 33	**33**
`F`	70	− 33	**37**
`H`	72	− 33	**39**

품질 문자	ord(문자)	− 33	Phred 점수
J	74	− 33	**41**

5-3. 계산 과정 따라가 보기

실습 파일 SRR1553607_1.fastq의 **첫 번째 read**로 계산해 봅시다.

```
품질 문자열: BB@FFFFFHHHHHJJJJJJJJJJJJJJJJJJJJGHIJJJJ...DDDDDDDDDD (101글자)
```

1. 문자를 점수로 바꾼다: B→33, B→33, @→31, F→37, F→37, …
2. 101개 점수를 모두 더한다: 합계 = **3781**
3. 염기 개수(101)로 나눈다: 3781 ÷ 101 ≈ **37.44**

→ 이 read의 평균 품질은 약 **Q37**로, 매우 좋은 read입니다.

5-4. 전체 평균은 어떻게 구할까?

```
total = 0
for name, seq, qual in reads:
    q = mean_quality(qual)
    total = total + q

print("전체 read 평균 품질:", total / len(reads))
```

- read마다 mean_quality()로 평균을 구해 total에 더합니다.
- 마지막에 read 개수(len(reads))로 나눕니다.
- 즉, 결과는 **"read별 평균 품질"들의 평균**입니다.

---

6. 실제 실행 결과와 해석

6-1. 실행 결과

```
$ python3 src/qual_summary.py data/SRR1553607_1.fastq
전체 read 평균 품질: 33.4497222958299

$ python3 src/qual_summary.py data/SRR1553607_2.fastq
전체 read 평균 품질: 30.830840894308746
```

파일	read 수	전체 평균 품질
SRR1553607_1.fastq (R1)	203,445	약 **33.45**
SRR1553607_2.fastq (R2)	203,445	약 **30.83**

(read 수 = 파일 줄 수 813,780 ÷ 4)

6-2. 해석

- **두 파일 모두 평균 Q30 이상**입니다. 평균적으로 염기 1,000개 중 1개 정도만 틀린다는 뜻이므로, 전체적으로 **품질이 좋은 데이터**라고 볼 수 있습니다.
- `_1`과 `_2`는 **페어드 엔드(paired-end)** 시퀀싱으로 같은 DNA 조각을 양쪽 끝에서 읽은 것입니다.
- **R2(`_2`)가 R1(`_1`)보다 품질이 조금 낮습니다** (33.45 → 30.83). Illumina 시퀀싱에서는 두 번째 read가 첫 번째 read보다 품질이 약간 떨어지는 일이 흔하므로, 이상한 결과는 아닙니다.

6-3. 이 결과만으로 알 수 없는 것

이 프로그램은 **숫자 하나(전체 평균)**만 보여 주기 때문에 다음은 알 수 없습니다.

- read의 **뒷부분**으로 갈수록 품질이 떨어지는지
- 품질이 아주 나쁜 read가 **몇 개나** 섞여 있는지

평균이 좋아도 일부 read나 read 끝부분은 품질이 낮을 수 있습니다. 더 자세히 보려면 FastQC 같은 도구로 위치별 품질 그래프를 확인하는 것이 좋습니다.

요약

항목	내용
입력	FASTQ 파일 1개
출력	전체 read 평균 품질 (화면 출력)
핵심 공식	점수 = `ord(문자) - 33` (Phred+33)
실습 결과	R1 ≈ 33.45, R2 ≈ 30.83 → 둘 다 Q30 이상, 양호